LAB 2

PRODUCTION INFERENCE ACCELERATION AND BATCH PROCESSING FOR EDGE YOLO26 DEPLOYMENTS

Course	DDM501
Weight	15%
Format	Team Lab (3-4 members per team)
Prerequisites	Lab 1 completed

1. OVERVIEW

1.1. Introduction

In Lab 1, you successfully split your Automated Parking Car Detection System into a decoupled microservice architecture: an I/O-bound StreamIngestion service and a compute-bound ObjectDetectionInference service communicating over a low-latency gRPC channel.

However, as deployments scale from a single entryway camera to dozens of continuous, high-definition IP cameras across a multi-level parking deck, a single-frame sequential CPU inference design fails. Under heavy stream loads, gateways experience severe queue delays, high latency spikes, and frame drops.

The objective of this lab is to introduce **Inference Acceleration and Batching**—the engineering practices required to maximize model throughput and minimize latency on target deployment hardware. You will compile your custom YOLO26 model into hardware-optimized formats, leverage hardware-specific runtimes (GPU/VPU/CPU instructions), and implement a highly efficient batch processing pipeline.

1.2. Scenario: Parking Facility Scale-up

Your automated parking operator client is expanding the system. Instead of 1 camera, the edge gateway must now process feeds from **8 concurrent IP cameras** situated at various parking sectors.

To support this demand, the gateway hardware has been updated with either an edge GPU (e.g., NVIDIA Jetson / RTX) or an optimized Intel/AMD CPU with vector acceleration extensions. Your task is to:

1. **Accelerate the Model Runtime:** Convert your custom PyTorch YOLO26-Nano weights (best.pt) to an optimized format tailored to your target execution platform:
   - **NVIDIA GPU Targets:** Compile to a **TensorRT** optimized engine (.engine).
   - **Intel CPU/GPU Targets:** Compile to an **OpenVINO** intermediate representation (.xml/.bin).
   - **Cross-Platform Fallbacks:** Compile to an **ONNX Runtime** optimized session using CUDA or CPU Execution Providers.
2. **Implement Batch Processing:** Upgrade your gRPC protocol from a single-frame request architecture to a **batched frame processing pipeline** that processes multiple images simultaneously.
3. **Analyze Performance:** Conduct comprehensive load testing to chart the latency vs. throughput trade-off curves as a function of batch size B and precision quantization (FP32 vs. FP16).

2. BACKGROUND

2.1. Library Runtime Acceleration

Standard PyTorch models execute using a dynamic computation graph. While highly flexible during training, this introduces significant overhead during inference due to python bindings, unoptimized layers, and redundant memory copying.

Production edge deployments compilation fuses layers and custom-tunes execution kernels for specific hardware:

- **NVIDIA TensorRT:** TensorRT is a highly optimized SDK for deep learning inference on NVIDIA GPUs. It performs:
  - **Layer and Tensor Fusion:** Combines adjacent nodes (like Convolution, Bias, and ReLU activation) into single execution kernels, reducing memory read/write cycles.
  - **Kernel Auto-Tuning:** Benchmarks hundreds of low-level CUDA kernels on the physical hardware to select the fastest configuration.
  - **Precision Calibration:** Safely downscales weight matrices from standard FP32 (32-bit floating point) to FP16 (half-precision) or INT8 (8-bit integer quantization) with minimal loss in mean Average Precision.
- **Intel OpenVINO:** The OpenVINO (Open Visual Inference and Neural Network Optimization) toolkit optimizes models on Intel CPUs, integrated GPUs, and VPUs. It exploits hardware instructions such as AVX-512 (Advanced Vector Extensions) and VNNI (Vector Neural Network Instructions) to perform parallel execution on the CPU.
- **ONNX Runtime (ORT):** A standard, cross-platform engine developed by Microsoft that executes compiled Open Neural Network Exchange format models via specialized *Execution Providers* (EPs) like CPU, CUDA, TensorRT, or DirectML.

2.2. Hardware Acceleration with GPUs

CPUs are optimized for low-latency execution of sequential tasks (relying on large cache sizes and complex control logic). GPUs, conversely, are designed for massive data-parallel architectures, featuring thousands of simple arithmetic logic units (ALUs) working in parallel.

For object detection, passing a single 640x640 frame through YOLO26 on a GPU fails to fully saturate the GPU's compute capacity. The GPU spending more time waiting for memory access than conducting matrix multiplication. Thus, to unlock hardware capability, **Batching** is essential.

2.3. Batch Processing Theory & Trade-Offs

Batching is the process of grouping multiple independent inference requests together into a single tensor payload of shape. By batching frames, the overhead of transferring data from the host memory (RAM) to the device memory (VRAM) is amortized, and the GPU's tensor cores can execute matrix multiplications in parallel.

3. HANDS-ON GUIDES

Task 1: Environment Setup & Optional Acceleration Runtimes

1. **Identify Local Specs:** Determine whether your lab computer has an NVIDIA GPU (runs TensorRT/CUDA) or an Intel/AMD CPU (runs OpenVINO/ONNX).
2. **Update Dependency Matrix:** Install the required platform bindings (tensorrt, openvino, or onnxruntime).
3. **Refactor Protobuf:** Modify protos/detector.proto to support batched frame arrays.

Task 2: Model Compilation and Optimization

1. **Write Compilation Script:** Create export_accelerated.py to compile your custom weights (best.pt) to TensorRT (.engine), OpenVINO (.xml), or ONNX (.onnx).

2. **Quantization Tuning:** Export your model with FP16 precision to double edge performance with minimal structural information loss.

Task 3: Develop the Batched gRPC Server

1. **Refactor Server:** Implement an updated gRPC server (inference_service/server_accelerated.py) that accepts list structures containing multiple raw image byte streams.
2. **Load Accelerated Engine:** Update inference_service/detector_accelerated.py to automatically bind and process inputs using the compiled engine formats.

Task 4: Develop the Multi-Stream Ingestion Pipeline

1. **Simulate Multi-Camera Ingestion:** Update stream_ingestion/ingest_batched.py to simulate reading from **multiple virtual cameras**.
2. **Implement Static Client-Side Batching:** Write buffer logic to queue frames from different cameras and dispatch them in a single gRPC batch request.

Task 5: Containerize and Benchmark

1. **Build Accelerated Docker Images:** Create Dockerfiles configured to support hardware drivers (such as CUDA and OpenVINO runtime libraries).
2. **Execute Benchmark Load Tests:** Run load tests and log latency vs. throughput metrics under varying batch sizes.

4. INSTRUCTION STEPS

a) 4.1. Adjust Batched Protocol Contract (protos/detector.proto)

b) 4.2. Model Compiler Script (export_accelerated.py): Create this utility to compile PyTorch model weights to optimized, half-precision (FP16) binaries or INIT8

c) 4.3. Accelerated Inference Core (inference_service/detector_accelerated.py): Implement the inference engine to dynamically detect and load whichever compiled format is available.

d) 4.4. Batched gRPC Server (inference_service/server_accelerated.py): Implement the batched gRPC server, ensuring that empty inputs are handled cleanly:

e) 4.5. Acceleration Target Container (Dockerfile.cpu): Create a Dockerfile configured to run optimized CPU workloads using OpenVINO and ONNX Runtime

5. DELIVERABLES & GRADING

5.1. Deliverables

Teams must submit their complete code repository, including:

1. **Model Compilation & Export Logs:**
   - Script export_accelerated.py used to compile weights.
   - Model binaries corresponding to your hardware configuration (best.engine, best.onnx, or best_openvino_model/).
2. **Refactored gRPC Protocol Contracts:**
   - Compiled proto configuration files under protos/.
3. **Optimized Microservice Stack:**

    - inference_service/ hosting the batched inference wrapper and gRPC server.
    - stream_ingestion/ configured for multi-stream ingestion and client-side batching.
4. **Automated Stress Benchmark Reports:**
    - Completed profiling metrics (latency vs. batch size) logged as table structures inside README.md.
    - Passing tests/test_benchmark.py testing assertions.

5.2. Grading Rubric

Criteria	Weight	Detailed Description
Model Optimization & Export	**25%**	Correct export implementation inside export_accelerated.py (10%), successfully generating half-precision compiled binaries (10%), verification of FP16 runtime size vs. FP32 (5%).
Batched Inference Server	**25%**	Server correctly parses multi-frame array requests (10%), dynamic loading of the highest performance compiled engine available (10%), robust validation of batch inputs and error-handling (5%).
Multi-Stream Ingestion Pipeline	**20%**	Resilient ingestion simulator simulating multiple distinct video channels (10%), implementation of an efficient client-side buffer queue to bundle frames before transmission (10%).
Stress Benchmarking & Testing	**20%**	Comprehensive benchmark test cases configured under Pytest (10%), load latency assertions pass within boundaries (5%), dynamic plotting or tabulating of FPS vs. Batch Size in reports (5%).
Documentation & Quality	**10%**	Clear instructions in README.md detailing hardware spec dependencies, docker setup instructions, and

		deployment guides (10%).

5.3. Submission Instructions

- **Deadline:** 1 week following the lab session.
- **Format:** Code package in standard .zip format or private GitHub repository link with viewer access granted to your course instructor.